



PetFood Mart

寵物食品線上商店 — 期末報告

第11組：林澤勳，許皓翔

GitHub: <https://github.com/sam0418/my-pet-shop>

Vue 3

Node.js

PostgreSQL

Docker

專案介紹



名稱：

PetFood Mart 寵物食品專賣



目標：

完整電商平台功能：商品瀏覽、購物車、訂單送出、管理後台



語言：

中英文雙語切換



下單：

WhatsApp 下單 + PDF 收據下載



管理：

商品 CRUD、運費設定

專案展示

PLACEHOLDER
放前端畫面

PLACEHOLDER
放前端畫面

技術棧



前端

- Vue 3 (CDN 模式)
- Tailwind CSS
- FontAwesome 6
- jsPDF + jspdf-autotable



後端

- Node.js / Express
- PostgreSQL 15
- JWT 身份驗證
- bcryptjs 密碼加密
- Joi 請求驗證
- express-rate-limit 限速
- Morgan 日誌

系統架構



- 前端：index.html 使用 Vue 3 CDN，透過 Fetch API 溝通後端
- 後端：Node.js / Express，負責業務邏輯與資料庫存取
- 資料庫：PostgreSQL 15，以 Docker 容器運行
- 部署：docker compose up -d 一鍵啟動所有服務

前端開發

```
<script type="module">
  import { isSupabaseConfigured, SUPABASE_CONFIG } from './js/config.js';
  import {
    DEFAULT_SETTINGS, ADMIN_CREDENTIALS, PICKUP_DISCOUNT_RATE,
    SHIPPING_METHODS, VIEWS, ADMIN_TABS, TOAST_TYPES, TOAST_DURATION
  } from './js/utils/constants.js';
  import {
    calculatePrice, cleanPhoneNumber, validateRequired, generateOrderNumber
  } from './js/utils/helpers.js';
  import { useTranslation } from './js/composables/useTranslation.js';
  import { useToast } from './js/composables/useToast.js';
  import { useCart } from './js/composables/useCart.js';
  import { useProducts } from './js/composables/useProducts.js';
  import { useAdmin } from './js/composables/useAdmin.js';
  import { useOrder } from './js/composables/useOrder.js';
  import { supabaseService } from './js/services/supabaseService.js';
  import { pdfService } from './js/services/pdfService.js';
```

```
▼ js
  ▼ composables
    JS useAdmin.js
    JS useCart.js
    JS useOrder.js
    JS useProducts.js
    JS useToast.js
    JS useTranslation.js
  ▼ services
    JS pdfService.js
    JS supabaseService.js
  ▼ utils
    JS constants.js
    JS helpers.js
```

前端開發

JS useProducts.js

```
import { ref, computed, reactive, onMounted } from 'vue';
import { supabaseService } from '../services/supabaseService.js';

export const useProducts = () => {
  const products = ref([]);
  const searchQuery = ref('');
  const isLoading = ref(false);

  const productForm = reactive({
    id: null,
    name: '',
    price: 0,
    stock: 0,
    discount: 0,
    image: '',
    description: ''
  });
};
```

```
const loadProducts = async () => {
  isLoading.value = true;
  try {
    const data = await supabaseService.fetchProducts();
    products.value = data || [];
  } catch (error) {
    console.error('Failed to load products:', error);
  } finally {
    isLoading.value = false;
  }
};

const saveProduct = async (payload) => {
  if (isEditing.value) {
    return await supabaseService.updateProduct(productForm.id, payload);
  } else {
    return await supabaseService.createProduct(payload);
  }
};

const deleteProduct = async (id) => {
  return await supabaseService.deleteProduct(id);
};
```

資料庫設計 (PostgreSQL 15)

products

商品主表

- id PK
- name (唯一)
- price
- stock
- discount
- image
- description
- created_at / updated_at

app_settings

網站設定

- id PK ('main')
- shipping_fee
- free_shipping_threshold
- whatsapp_number
- updated_at

orders

訂單表

- id PK
- order_number (唯一)
- customer_name/phone/e mail
- total_amount
- items (JSONB)
- status
- created_at / updated_at

資料庫架構

ORDERS			
int	id	PK	訂單ID (SERIAL)
varchar_50	order_number	UK	訂單編號 (UNIQUE)
decimal	total_amount		總金額
int	customer_id	FK	指向唯一id
jsonb	items		商品快照陣列 (包含 Product_ID)
varchar_20	status		訂單狀態
timestamp	created_at		
timestamp	updated_at		

PRODUCTS			
int	id	PK	商品ID (SERIAL)
varchar_255	name	UK	商品名稱 (UNIQUE)
decimal	price		價格 (>=0)
int	stock		庫存 (>=0)
decimal	discount		折扣 (0~100)
text	image		圖片網址
text	description		描述
timestamp	created_at		
timestamp	updated_at		

CUSTOMERS			
int	id	PK	顧客ID (SERIAL)
varchar_255	customer_name		顧客姓名
varchar_20	customer_phone	UK	顧客電話(UNIQUE)
varchar_255	customer_email	UK	顧客Email(UNIQUE)
timestamp	created_at		
timestamp	updated_at		

- 資料庫將表格主要拆為訂單、商品和顧客
- 訂單中的商品快照以JSONB格式儲存

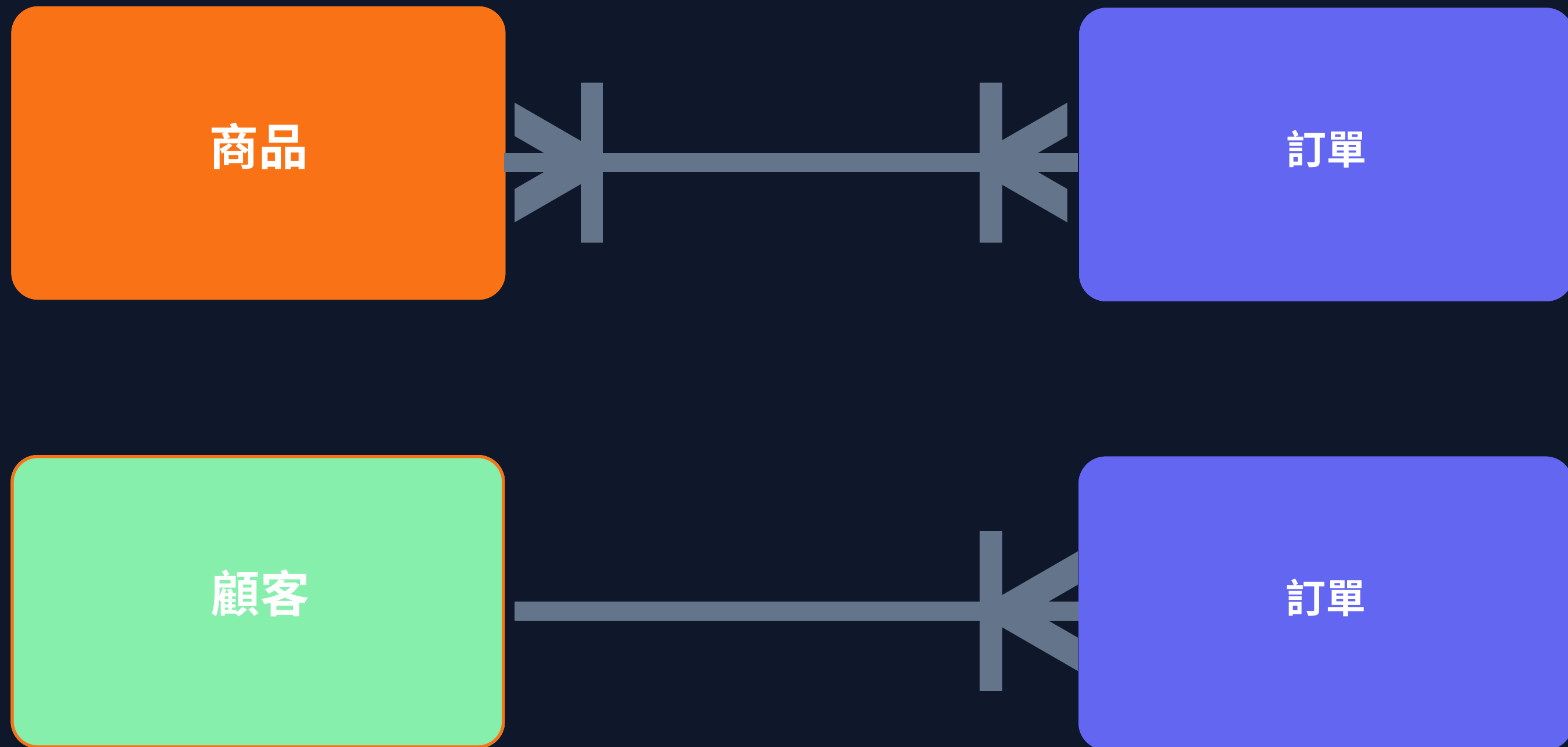
資料庫架構

```
-- 商品表
✓ CREATE TABLE IF NOT EXISTS products (
  id SERIAL PRIMARY KEY, --unique product id
  name VARCHAR(255) NOT NULL UNIQUE,
  price DECIMAL(10, 2) NOT NULL CHECK (price >= 0),
  stock INTEGER NOT NULL DEFAULT 0 CHECK (stock >= 0),
  discount DECIMAL(5, 2) NOT NULL DEFAULT 0 CHECK (discount >= 0 AND discount <= 100),
  image TEXT,
  description TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

JSONB Example :

```
{
  "product_id": int,
  "quantity": int,
  "discount": int (range 0~100)
}
```

資料庫架構



資料庫架構

- 用於設定整體平台設定
- 也可只在特定範圍內使用

APP_SETTINGS			
varchar_50	id	PK	設定識別碼
decimal	shipping_fee		基本運費
decimal	free_shipping_threshold		免運門檻
varchar_20	whatsapp_number		客服電話
timestamp	created_at		
timestamp	updated_at		

核心功能



商品瀏覽

分頁、排序、搜尋



購物車

新增、調整數量、刪除商品



配送方式

自取 / 宅配 / 自訂配送



WhatsApp

WhatsApp 下單整合



PDF 收據

含訂單流水號，自動生成



管理後台

商品 CRUD + 運費設定



多語支援

中英文一鍵切換



身份驗證

JWT + Rate Limiting 保護

REST API 設計



公開端點（無需驗證）

- GET /api/products
- GET /api/products/:id
- GET /api/settings
- POST /api/auth/login
- GET /health



受保護端點（需 Bearer Token）

- POST /api/products
- PUT /api/products/:id
- DELETE /api/products/:id
- POST /api/settings



安全機制

- JWT 驗證（24小時過期）
- Rate Limit：每 15 分鐘 100 次請求 / 登入最多 5 次
- CORS 白名單控制
- Joi 輸入驗證防 SQL Injection

REST API 設計

GET	/api/products	否	取得商品列表（支援分頁、排序）
GET	/api/products/:id	否	取得單一商品
POST	/api/products	是	新增商品
PUT	/api/products/:id	是	更新商品
DELETE	/api/products/:id	是	刪除商品
GET	/api/settings	否	取得網站設定
POST	/api/settings	是	更新網站設定
POST	/api/auth/login	否	管理員登入，回傳 JWT
GET	/health	否	健康檢查

後端核心功能

JWT 身份驗證：

- Token 有效期：24 小時
- Header 格式：
Authorization: Bearer
<token>
- Token 包含：{ id,
username, role }

```
* 驗證 JWT 令牌
*/
const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // Bearer TOKEN

  if (!token) {
    return res.status(401).json({ error: 'Access token required' });
  }

  jwt.verify(token, JWT_SECRET, (err, user) => {
    if (err) {
      return res.status(403).json({ error: 'Invalid or expired token' });
    }
    req.user = user;
    next();
  });
};
```


後端核心功能

Rate Limiting:

- 一般 API: 每個 IP 每 15 分鐘最多 100 次請求
- 登入端點: 每個 IP 每 15 分鐘最多 5 次失敗嘗試
- 開發端檢查 /health: 跳過限速

```
// ===== 請求限速 =====  
const limiter = rateLimit({  
  windowMs: 15 * 60 * 1000, // 15分鐘  
  max: 100, // 最多100個請求  
  message: 'Too many requests from this IP, please try again after 15 minutes',  
  standardHeaders: true,  
  legacyHeaders: false,  
  skip: (req) => req.path === '/health' // 不限速檢查  
});  
  
const authLimiter = rateLimit({  
  windowMs: 15 * 60 * 1000,  
  max: 5, // 登入最多5次  
  skipSuccessfulRequests: true,  
  message: 'Too many login attempts, please try again after 15 minutes'  
});  
  
app.use('/api/', limiter);  
app.use('/api/auth/login', authLimiter);
```

後端核心功能

```
// 驗證參數
if (page < 1) page = 1;
if (limit < 1) limit = 1;
if (limit > 100) limit = 100; // 防止過大查詢

// 驗證排序欄位 (防止SQL注入)
const allowedSortFields = ['id', 'name', 'price', 'stock', 'discount', 'created_at', 'updated_at'];
if (!allowedSortFields.includes(sort)) {
  return res.status(400).json({ error: 'Invalid sort field' });
}

if (!['ASC', 'DESC'].includes(order)) {
  return res.status(400).json({ error: 'Invalid order value. Use ASC or DESC' });
}
```

輸入驗證：

- 使用 Joi 驗證所有 Request Body
- 排序欄位白名單防止 SQL Injection (allowedSortFields)
- ID 欄位驗證為數字型態

後端核心功能

採用記憶體快取 (in-memory cache) 減少資料庫查詢次數：

```
/**
 * 設置快取
 * @param {string} key - 快取鍵
 * @param {*} value - 快取值
 * @param {number} ttl - 生存時間 (秒)
 */
set(key, value, ttl = 300) {
  // 清除舊的計時器
  if (this.timers.has(key)) {
    clearTimeout(this.timers.get(key));
  }

  // 設置快取
  this.cache.set(key, {
    value,
    createdAt: Date.now(),
    ttl: ttl * 1000
  });
}
```

```
/**
 * 獲取快取
 * @param {string} key - 快取鍵
 * @returns {*} 快取值或null
 */
get(key) {
  const cached = this.cache.get(key);
  if (!cached) return null;

  const age = Date.now() - cached.createdAt;
  if (age > cached.ttl) {
    this.cache.delete(key);
    return null;
  }

  return cached.value;
}
```

✓ 已完成功能

- 完整 CRUD 商品管理後台
- JWT 身份驗證流程
- Docker 容器化部署
- 自動產生模板化PDF收據
- Rate Limiting 防暴力攻擊
- 記憶體快取減少 DB 查詢
- WhatsApp 下單整合

💡 學習收穫

- 前後端分離架構實作
- RESTful API 設計原則
- PostgreSQL + JSONB 彈性儲存
- 安全性設計：驗證、限速、CORS
- Docker / docker-compose 容器化
- JWT 無狀態身份驗證機制



GitHub 專案

<https://github.com/sam0418/my-pet-shop>

- 完整原始碼（前端 + 後端）
- Dockerfile + docker-compose.yml
- init.sql 資料庫初始化腳本
- README.md 含完整啟動說明

感謝聆聽！